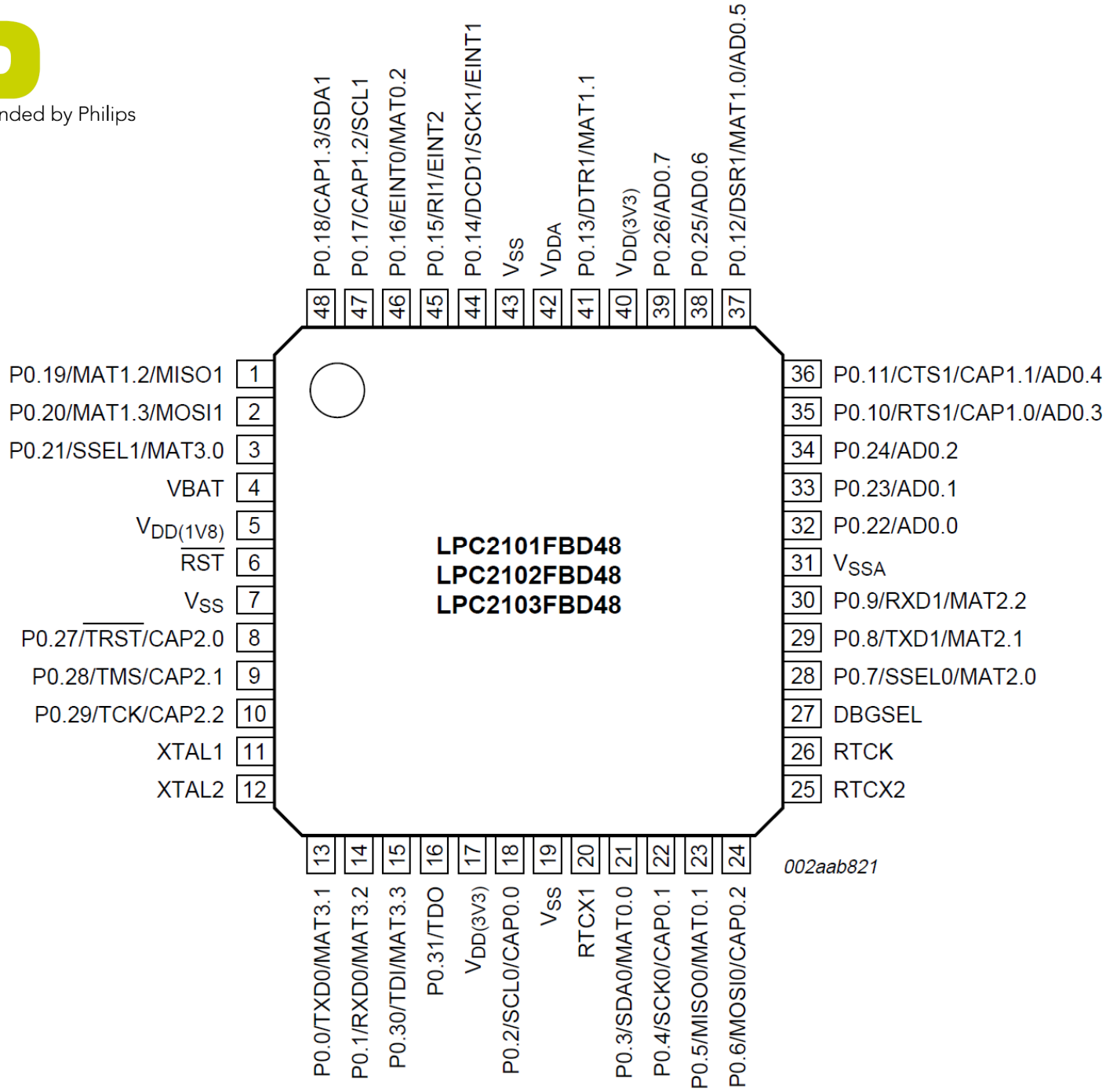
INDEX

- Introduction
- System Control Block
- GPIO
- Vectored Interrupt Controller (VIC)
- UART
- Timers

The LPC2103 microcontroller



The LPC2103 microcontroller

MAM: Memory Accelerator Module

AHB: Advanced High-performance Bus

APB: ARM Peripheral Bus

GPIO: General Purpose Input/Output

ADC: Analog-Digital Converter

PLL: Phase Locked Loop

VIC: Vectored Interrupt Controller

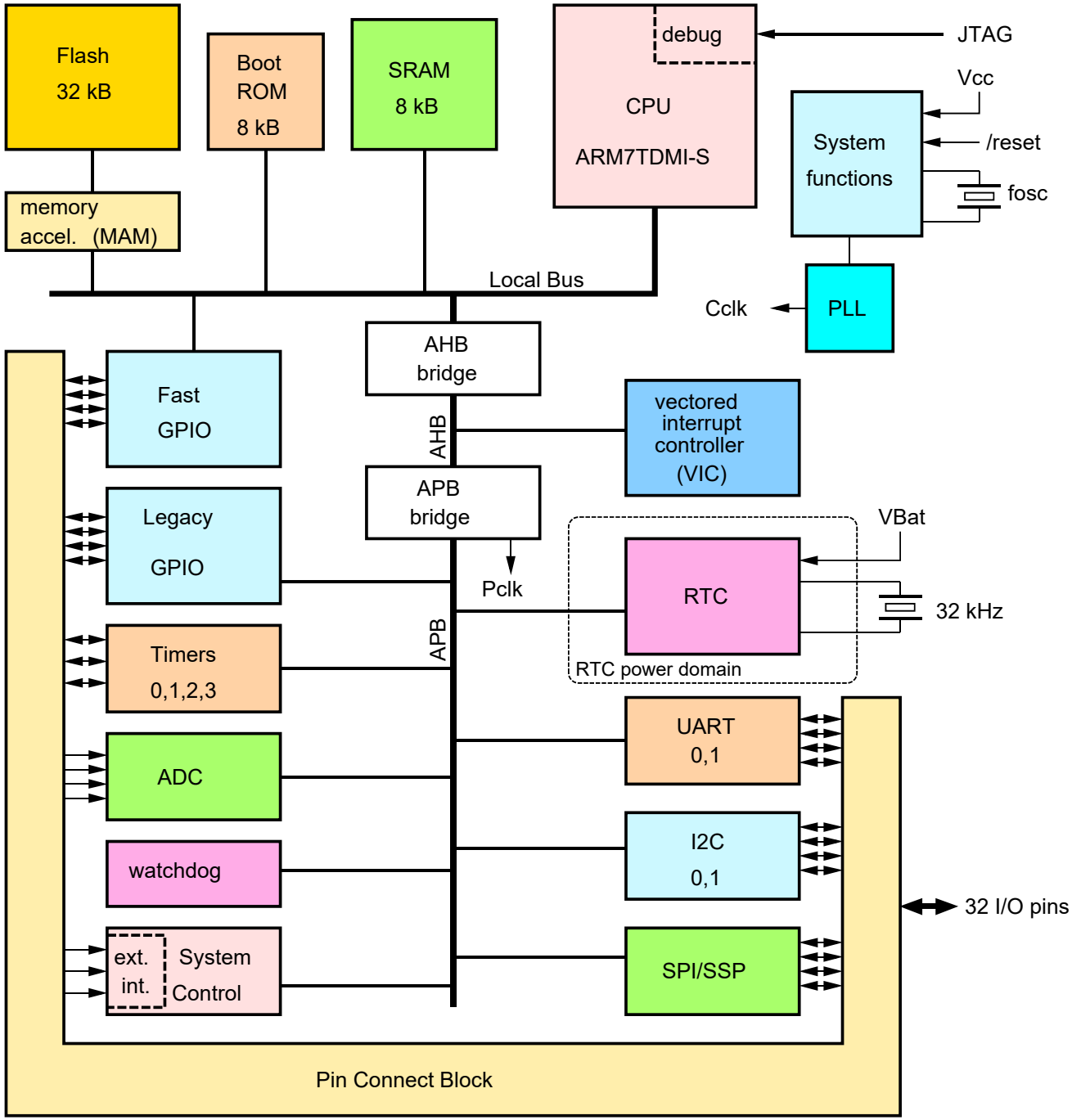
RTC: Real-Time Clock

UART: Universal Asynchronous Receiver-Transmitter

I2C: Inter Integrated Circuit (Phillips)

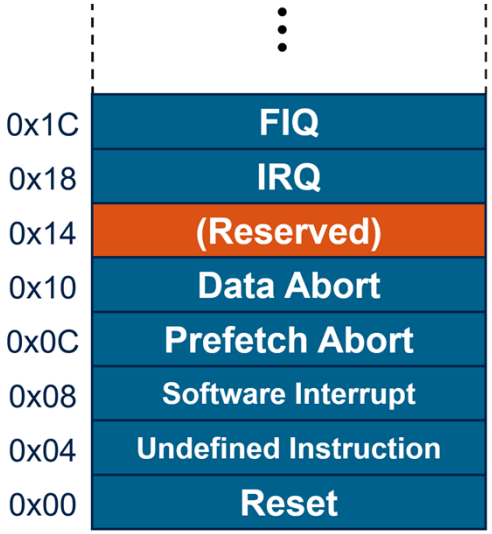
SSP: Synchronous Serial Port

SPI: Serial Peripheral Interface (Motorola)



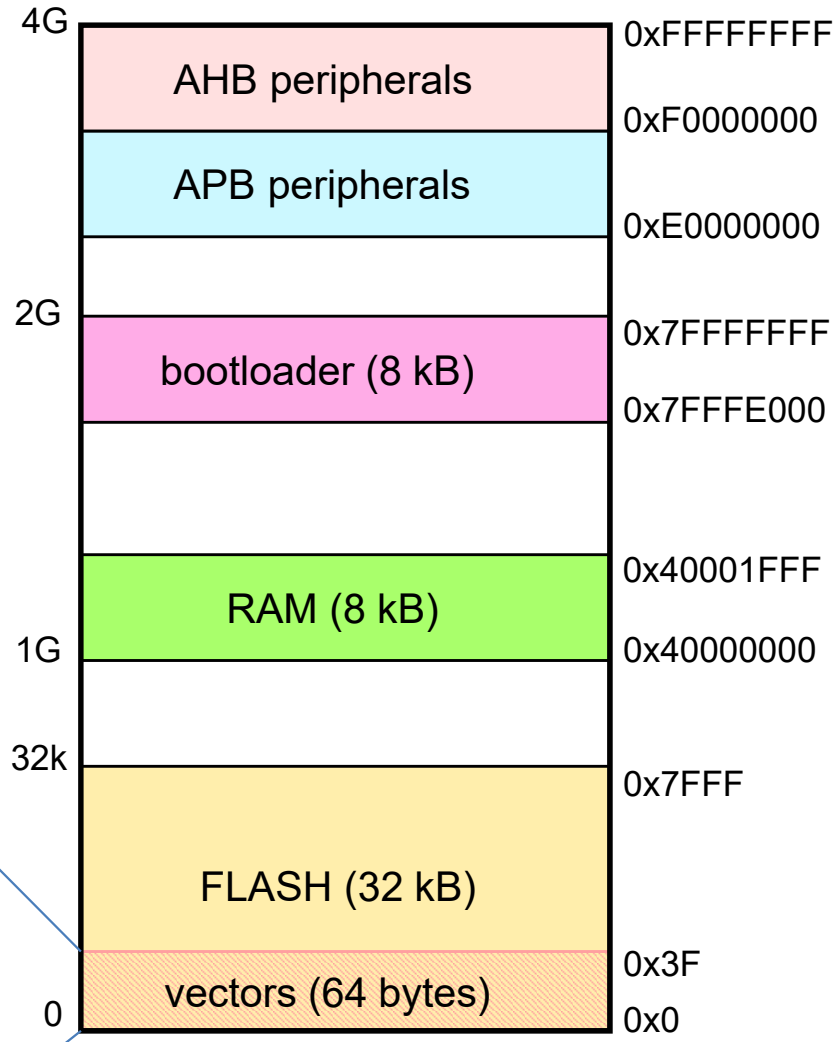
LPC2103. Memory map

- Memory mapped I/O
- Bootloader ROM for Flash programming
- User code can run from Flash or RAM



Vector Table

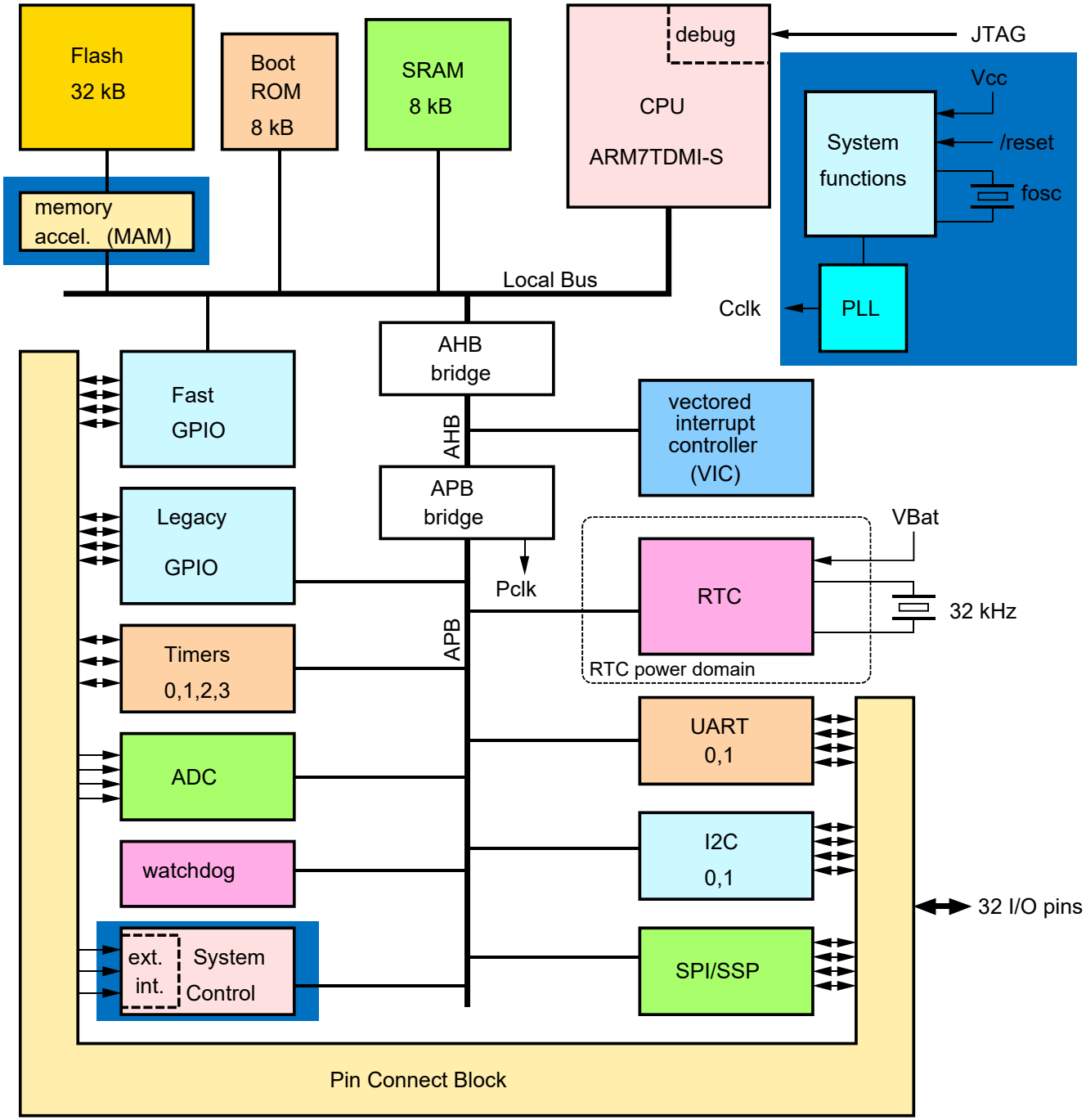
```
---- /*****************************************/
---- /** Vector Table & related constants  **/
---- /*****************************************/
----
0000 _vectors:        ldr     PC, Reset_Addr
0004                  ldr     PC, Undef_Addr
0008                  ldr     PC, SWI_Addr
000C                  ldr     PC, PAbt_Addr
0010                  ldr     PC, DAbt_Addr
0014                  nop
0018                  ldr     PC, [PC,#-0xFF0]
001C                  ldr     PC, FIQ_Addr
---- /*-----------------------------------------*/
---- Reset_Addr:      .word   Reset_Code
---- Undef_Addr:      .word   UNDEF_Routine
---- SWI_Addr:        .word   SWI_Routine
---- PAbt_Addr:       .word   PABORT_Routine
---- DAbt_Addr:       .word   DABORT_Routine
---- FIQ_Addr:        .word   FIQ_Routine
----
```



INDEX

- Introduction
- System Control Block
  - Clock generation
  - Power control
  - Vector mapping
  - Memory Accelerator
- GPIO
- Vectored Interrupt Controller (VIC)
- UART
- Timers

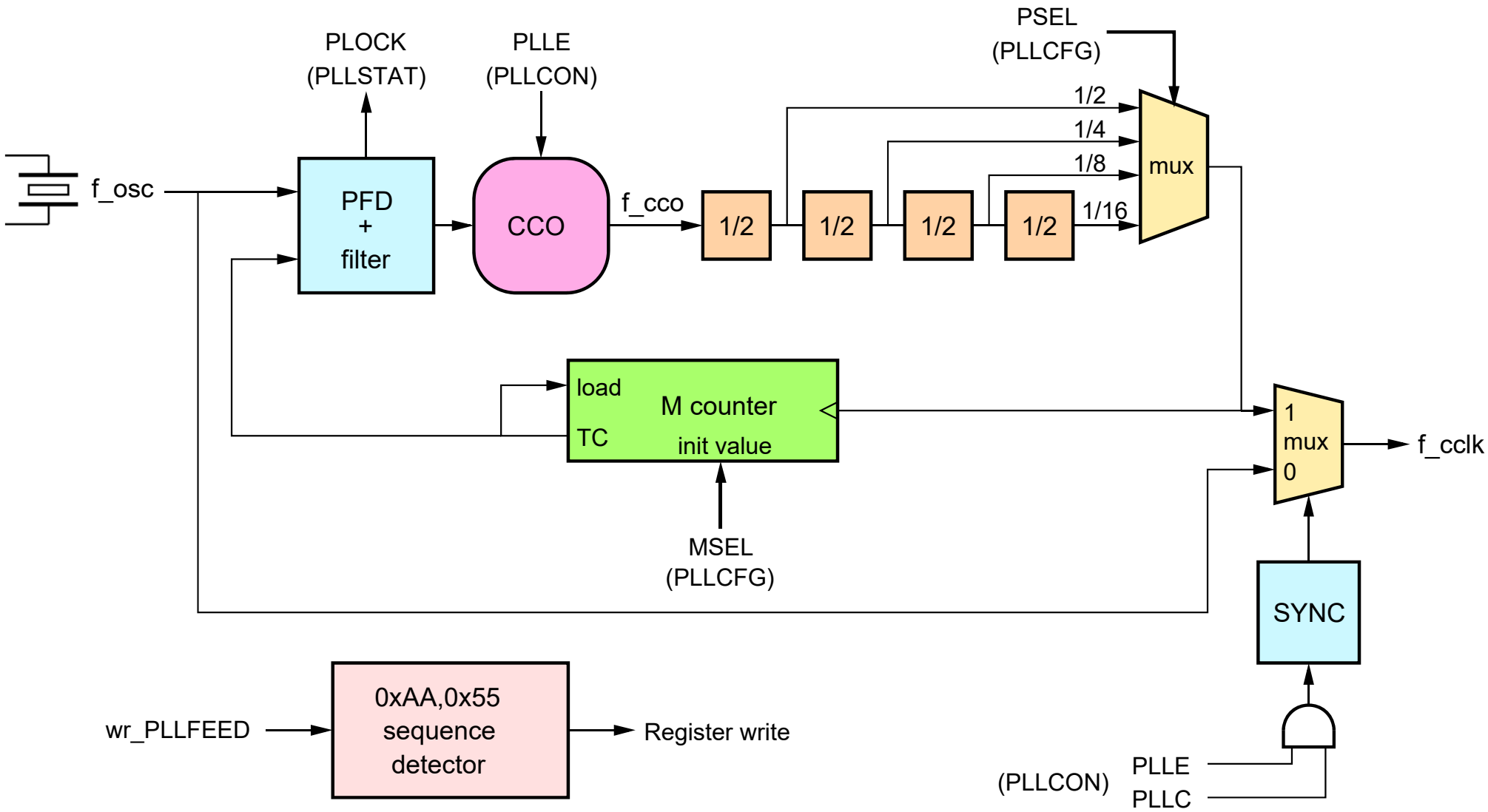
The LPC2103 microcontroller



LPC2103. System control block

- Controls several functions of the MCU
  - Clock generation (PLL, APB divider)
  - Power control (low power modes, peripheral power control)
  - External interrupts
  - Other controls (vector mapping,…)

LPC2103 System clock (PLL)



PFD: Phase-Frequency Detector
CCO: Current Controlled Oscillator

PLLE: PLL Enable
PLLC: PLL Connect

LPC2103 PLL registers

PLLCFG (WO) Configuration

7	6	5	4	3	2	1	0
-	PSEL		MSEL				

dir: (0xE01FC084)

PLLCON (WO) Control

7	6	5	4	3	2	1	0
-	-	-	-	-	-		

dir: (0xE01FC080)

PLLE

PLLC

PLLSTAT (RO) Status

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-	-	-	-	-				-	PSEL		MSEL				

dir: (0xE01FC088)

PLLE

PLOCK

PLLC

PLLFEED (WO) Security

7	6	5	4	3	2	1	0
KEY VALUE							

dir: (0xE01FC08C)

$f_cclk = f_osc * (MSEL + 1)$

$f_cco = f_cclk * 2^{(PSEL+1)}$

These conditions must be met:

10 MHz < f_osc < 25 MHz

f_cclk < 70 MHz

156 MHz < f_cco < 320 MHz

LPC2103 PLL turn-on procedure

- After Reset or a wakeup interrupt the microcontroller runs on the crystal oscillator
- To enable the PLL:
  1. Set PLLCFG with proper values for MSEL and PSEL
  2. Write 0xAA to PLLFEED, write 0x55 to PLLFEED
  3. Set PLLCON=1 (PLL enable on)
  4. Write 0xAA to PLLFEED, write 0x55 to PLLFEED
  5. Wait until bit 10 in PLLSTAT (PLOCK) becomes 1. If MSEL and PSEL have reasonable values this step would not take more than 100 µs
  6. Set PLLCON=3 (PLL enable on, PLL connect on)
  7. Write 0xAA to PLLFEED, write 0x55 to PLLFEED

LPC2103 PLL turn-on procedure

```
----                    /*****************************************************
----                    /* System init
----                    /* PLL x 4, PCLK x 1/4
----                    /* CCLK=58.9824MHz, PCLK=58.9824MHz
----                    /* UART0 at 115200 bps, 8-bit, 1-stop, no parity, FIFO on
----                    /*****************************************************
----
0150                    ldr             r1,=0xE01FC000
0154                    mov             r2,#0xaa
0158                    mov             r3,#0x55
015C                    mov             r0,#0x23                @ PLL x 4
0160                    strb    r0,[r1,#0x84]   @ PLLCFG
0164                    strb    r2,[r1,#0x8c]
0168                    strb    r3,[r1,#0x8c]
016C                    mov             r0,#1                   @ PLL on
0170                    strb    r0,[r1,#0x80]   @ PLLCON
0174                    strb    r2,[r1,#0x8c]
0178                    strb    r3,[r1,#0x8c]
017C 1:                 ldrh    r0,[r1,#0x88]   @ PLLSTAT
0180                    tst             r0,#0x400               @ PLOCK on ?
0184                    beq             1b
0188                    mov             r0,#3                   @ PLL connected
018C                    strb    r0,[r1,#0x80]   @ PLLCON
0190                    strb    r2,[r1,#0x8c]
0194                    strb    r3,[r1,#0x8c]
0198                    mov             r0,#1
019C                    str             r0,[r1,#0x100]  @ APBDIV: PCLK=CCLK/1
```

APB divider

- The peripheral bus (APB) can run at a lower clock frequency than the CPU (cclk)
- Register APBDIV selects the APB clock (pclk):

APBDIV[1:0]	pclk/cclk
00	1/4 (reset value)
01	1
10	1/2
11	reserved

Low power modes

- 3 operating modes
  - Normal all clocks running.
  - Idle CPU clock stopped, remaining clocks running. Any interrupt reverts to normal mode without delay.
  - Power Down all clocks stopped (except WD and RTC). Only wakeup interrupts revert to normal mode (with PLL disabled and after a several millisecond start-up). Microampere current consumption.
- Idle and Power down modes are entered by writing to register PCON:
  - Bit 0: IDL: writing 1 to this bit enters Idle mode.
  - Bit 1: PD: writing 1 to this bit enters power down mode.

(if PCON[1:0]=11 power down mode is entered)

Power control of peripherals

- Many peripherals can be turned off to save power.
- After reset all peripherals are powered.
- The register PCONP controls which peripheral is powered (bit at 1 means ON):

Bit	Periph.	Bit	Periph.	Bit	Periph.
1	TIMER0	7	I2C0	12	ADC
2	TIMER1	8	SPI0	19	I2C1
3	UART0	9	RTC	22	TIMER2
4	UART1	10	SPI1/SSP	23	TIMER3

- Unpowered peripherals do not hold the values written into their registers.

LPC2xxx Vector block mapping

- The first 32 bytes of the address space contains the ARM vectors for reset, interrupts and exceptions.
- Some space is also usually required for constants.
- In the LPC2xxx MCUs the first 64 bytes can be assigned to different memory areas through the MEMMAP register:

MEMMAP[1:0]	Memory	Accesses to 0x0 – 0x3F mapped to
00	Bootloader	0x7FFFE000 – 0x7FFFE03F
01	Flash	0x0 – 0x3F
10	RAM	0x40000000 – 0x4000003F
11	reserved	

- After reset the bootloader is always executed. If a valid code (correct checksum) is found in flash the vectors are mapped to flash and then a jump to address 0x0 is executed.

LPC2xxx Memory Accelerator Module (MAM)

- Flash memory is slow (50 ns access time). It takes 4 cycles per access for a 70 MHz CCLK.
- MAM speeds-up code execution from flash by fetching data 128 bits at a time (4 op-codes).
- Useful for sequential accesses (code execution).
- Registers
  - MAMCR MAM control register. Only bits [1:0] used:

00	MAM disabled.
01	MAM partially enabled (only sequential reads).
10	MAM fully enabled (like a small cache).
11	Reserved.

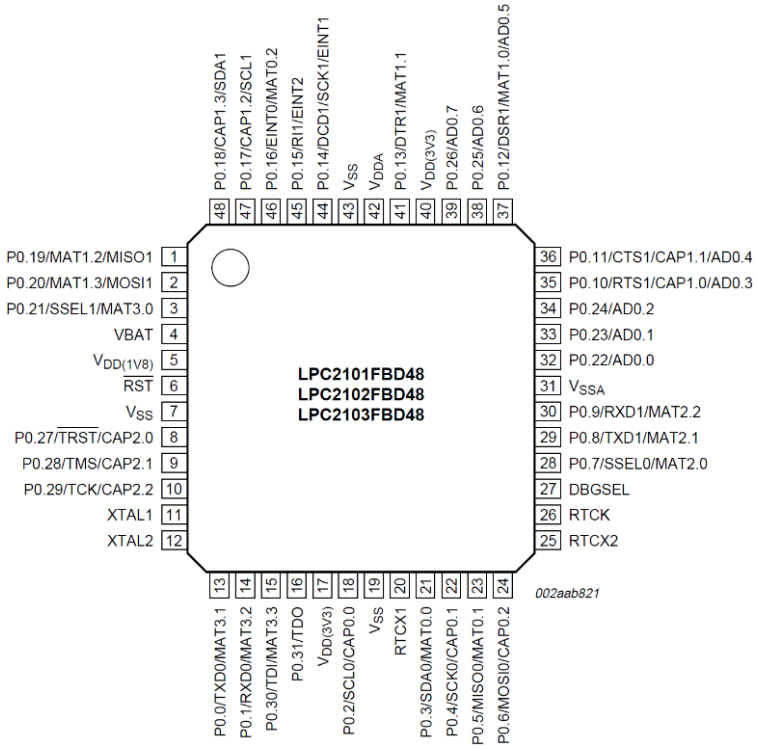
  - MAMTIM bits [2:0]: number of CCLK cycles per flash read.

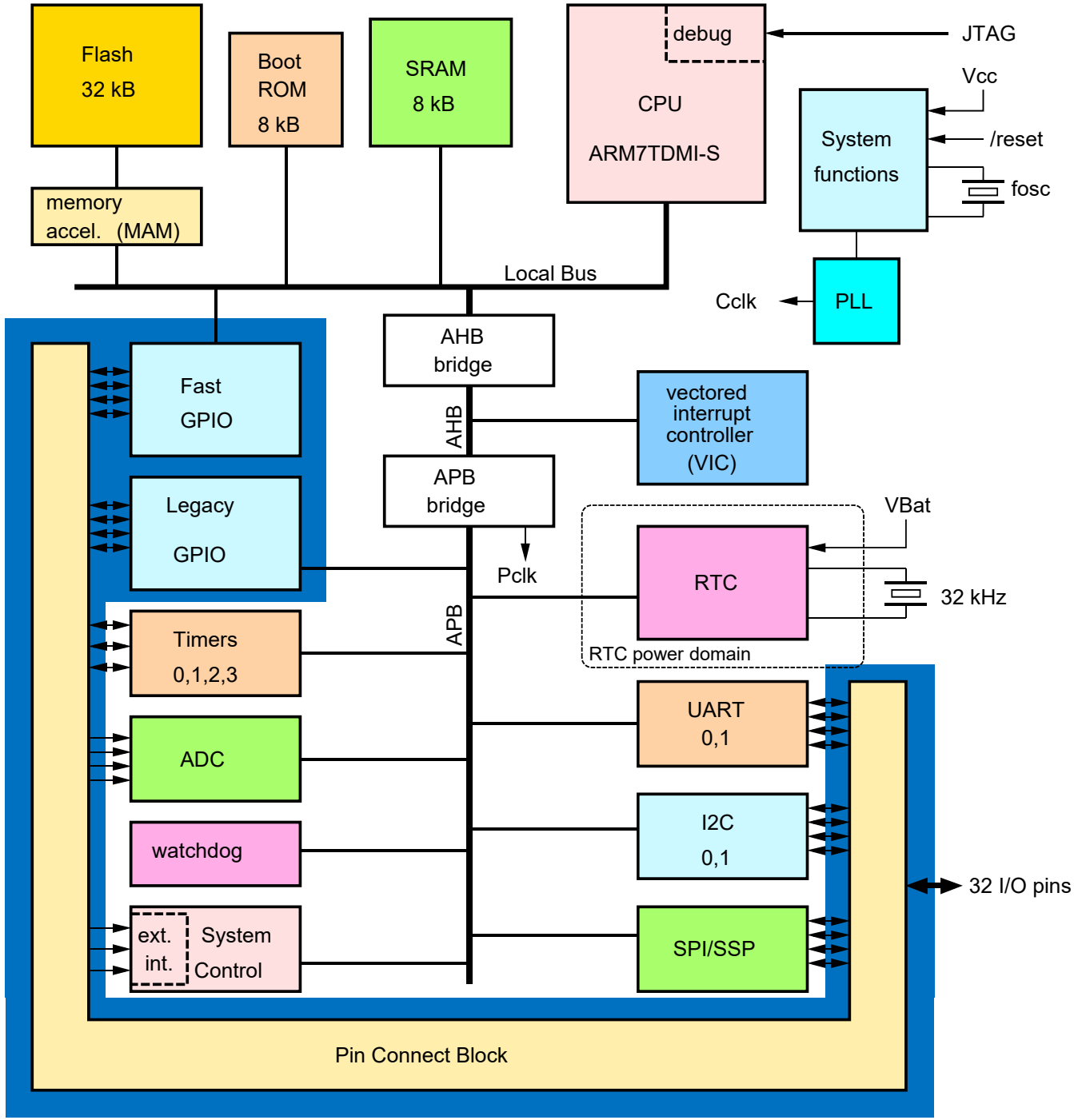
    It should be MAMTIM = f_cclk*50 ns (round to upper value. 000 is invalid).
- After reset MAM is disabled and 7 cycles per access are used.

INDEX

- Introduction
- System Control Block
- GPIO
  - Pin Connect Block
  - GPIO logic
- Vectored Interrupt Controller (VIC)
- UART
- Timers

The LPC2103 microcontroller





LPC2103 Pin Connect Block

- PINSELx registers select which peripheral is connected with each pin
- 2 bits per pin selects 4 possible connections:

PINSEL0 dir: (0xE002C000)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	9	8	7	6	5	4	3	2	1	0
Pin 15	Pin 14	Pin 13	Pin 12	Pin 11	Pin 10	Pin 9	Pin 8	Pin 7	Pin 6	Pin 5	Pin 4	Pin 3	Pin 2	Pin 1	Pin 0										

PINSEL1 dir: (0xE002C004)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	9	8	7	6	5	4	3	2	1	0
Pin 31	Pin 30	Pin 29	Pin 28	Pin 27	Pin 26	Pin 25	Pin 24	Pin 23	Pin 22	Pin 21	Pin 20	Pin 19	Pin 18	Pin 17	Pin 16										

- After reset all pins are selected as GPIO

LPC2103 Pin Connect Block. PINSEL0

PIN	Bits:	00	01	10	11
P0.0	1:0	GPIO	RXD0	MAT3.1	-
P0.1	3:2	GPIO	TXD0	MAT3.2	-
P0.2	5:4	GPIO	SCL0	CAP0.0	-
P0.3	7:6	GPIO	SDA0	MAT0.0	-
P0.4	9:8	GPIO	SCK0	CAP0.1	-
P0.5	11:10	GPIO	MISO0	MAT0.1	-
P0.6	13:12	GPIO	MOSI0	CAP0.2	-
P0.7	15:14	GPIO	SSEL0	MAT2.0	-
P0.8	17:16	GPIO	TXD1	MAT2.1	-
P0.9	19:18	GPIO	RXD1	MAT2.2	-
P0.10	21:20	GPIO	RTS1	CAP1.0	AD0.3
P0.11	23:22	GPIO	CTS1	CAP1.1	AD0.4
P0.12	25:24	GPIO	DSR1	MAT1.0	AD0.5
P0.13	27:26	GPIO	-	MAT1.1	DTR1
P0.14	29:28	GPIO	EINT1	SCK1	DCD1
P0.15	31:30	GPIO	EINT2	-	RI1

UART0
I2C0
SPI0
SPI1
UART1
A/D
Timers
Ext. INT

LPC2103 Pin Connect Block. PINSEL1

PIN	Bits:	00	01	10	11
P0.16	1:0	GPIO	EINT0	MAT0.2	-
P0.17	3:2	GPIO	SCL1	CAP1.2	-
P0.18	5:4	GPIO	SDA1	CAP1.3	-
P0.19	7:6	GPIO	MISO1	MAT1.2	-
P0.20	9:8	GPIO	MOSI1	MAT1.3	-
P0.21	11:10	GPIO	SSEL1	MAT3.0	-
P0.22	13:12	GPIO	-	-	AD0.0
P0.23	15:14	GPIO	-	-	AD0.1
P0.24	17:16	GPIO	-	-	AD0.2
P0.25	19:18	GPIO	-	-	AD0.6
P0.26	21:20	GPIO	-	-	AD0.7
P0.27	23:22	GPIO	TRST	CAP2.0	-
P0.28	25:24	GPIO	TMS	CAP2.1	-
P0.29	27:26	GPIO	TCK	CAP2.2	-
P0.30	29:28	GPIO	TDI	MAT3.3	-
P0.31	31:30	GPIO	TDO	-	-

I2C1
SPI1
A/D
Timers
Ext. INT
JTAG (debug)

LPC2xxx GPIO

- Fast or legacy GPIO
  - Same pins but different timings
  - Only one of these two peripherals can be active
    - Selectable from System Control Block register:

      GPIOM, bit 0: 0: Legacy GPIO 1: Fast GPIO
  - Legacy GPIO after reset
    - APB bus timings
    - Same interface as for older LPC2xxx microcontrollers
    - Four 32-bit registers per I/O port:

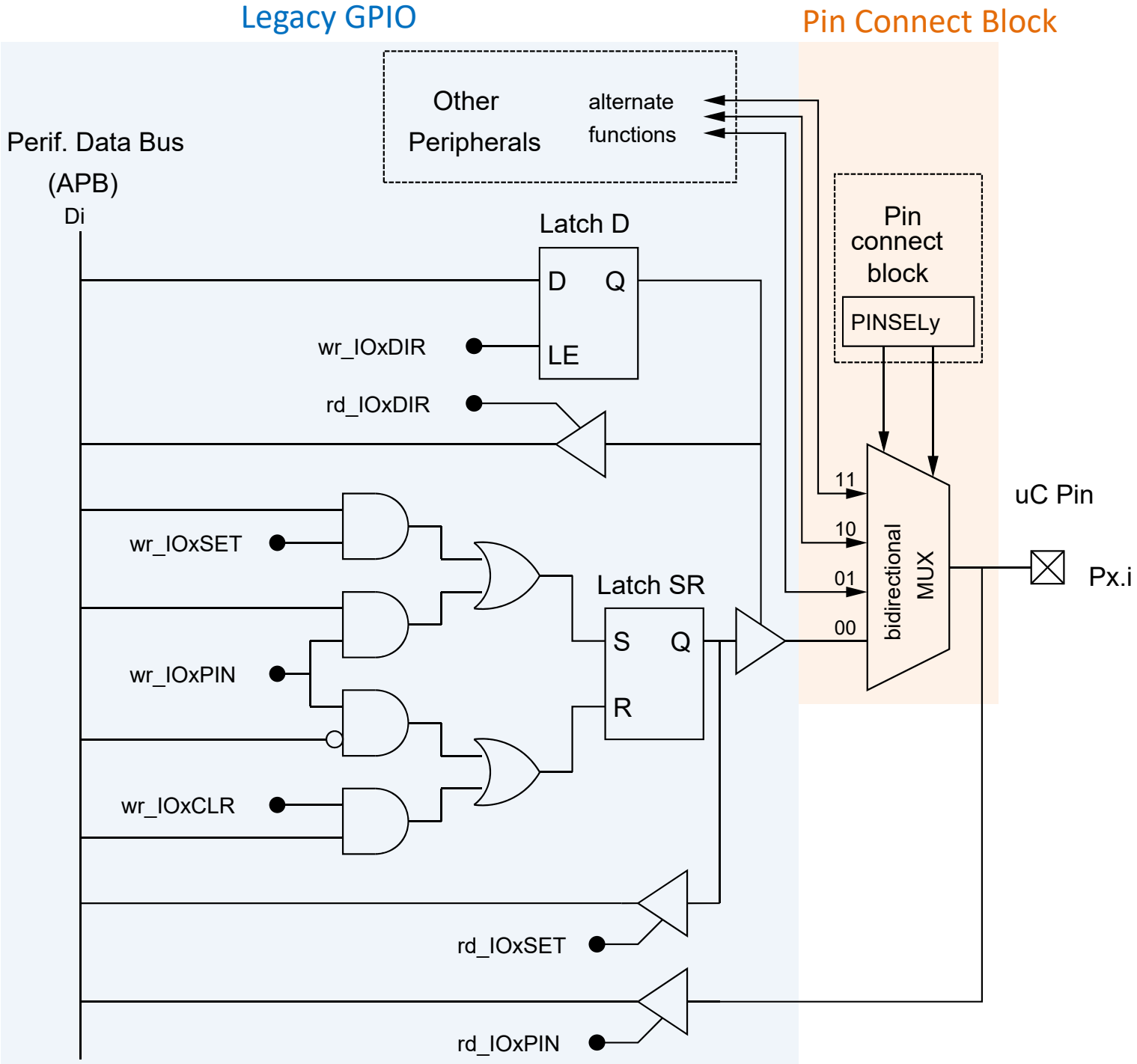
      IOxDIR (rw)

      IOxPIN (rw)

      IOxSET (rw)

      IOxCLR (wo) (replace 'x' with port number, for example: IO0DIR)
    - After reset all pins are selected as input

LPC2xxx. GPIO: pin diagram



LPC2xxx GPIO. Registers

- Registers (32-bit, one bit per pin)
  - IOxDIR (rw): Pin direction (dir: 0xE0028008)

    0: Input 1: Output
  - IOxPIN (rw): Pin Value (dir: 0xE0028000)

    Read: The bit reflects the logic level of the pin regardless of its direction or the function selected in Pin Connect Block

    Write: The level of the pin is changed to the written bit
  - IOxSET (rw): Pin Set (dir: 0xE0028004)

    Write: 1: Set pin (pin=1) 0: no action

    Read: The value of the output latch is returned
  - IOxCLR (wo): Pin Clear (dir: 0xE002800C)

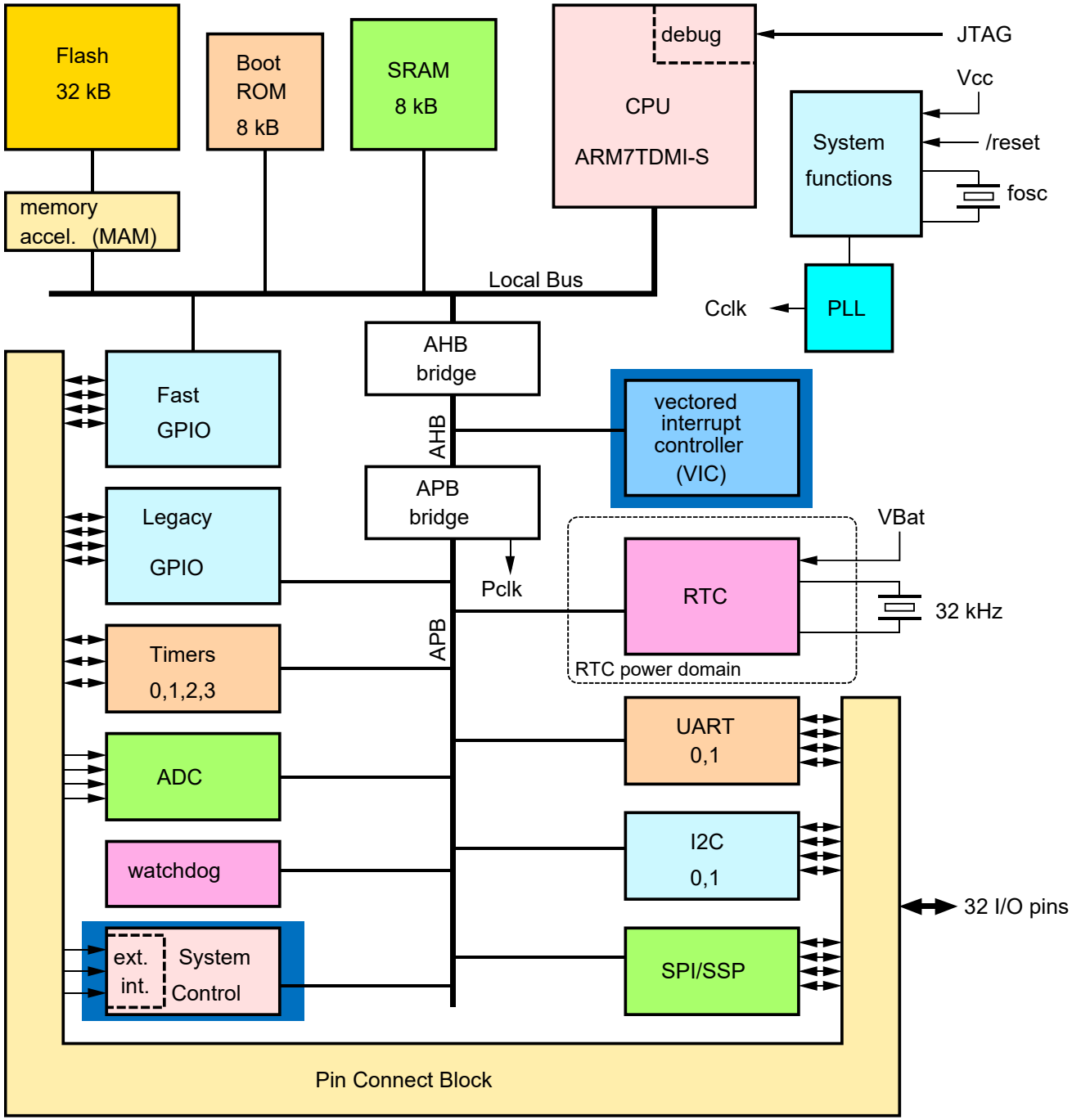
    Write: 1: Clear pin (pin=0) 0: no action

Only pins programmed as GPIO in the Pin Connect Block and as outputs in IOxDIR are actually changed by writes to IOxPIN, IOxSET and IOxCLR

INDEX

- Introduction
- System Control Block
- GPIO
- Vectored Interrupt Controller (VIC)
  - VIC
  - External interrupts
- UART
- Timers

The LPC2103 microcontroller

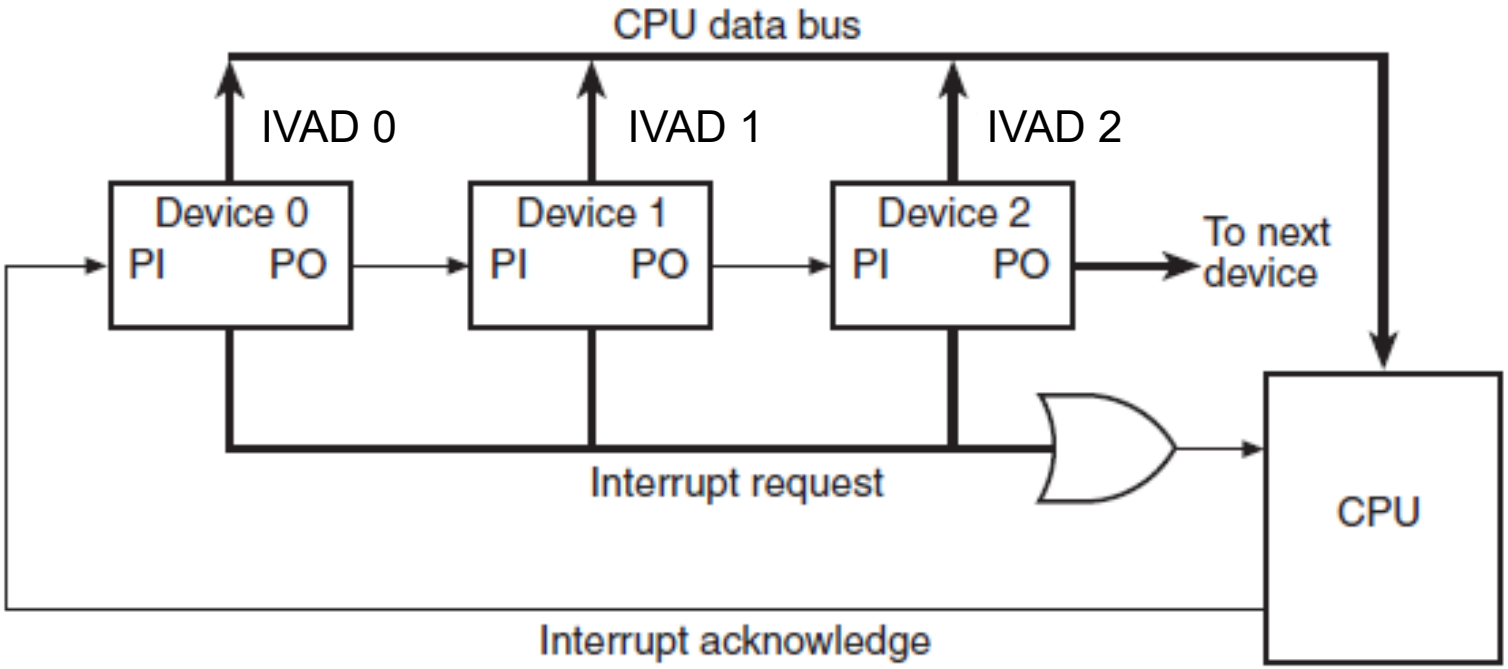


INTERRUPCIONES

Según cómo se llega al programa de servicio a interrupción (ISR):

- Interrupciones no vectorizadas – gestionadas por Software – Un único programa de servicio a interrupción decide cuál atender. Ventaja: programable. Desventaja: lento.

- Interrupciones vectorizadas – gestionadas por Hardware – Cada fuente de interrupción tiene asignado su propio vector de interrupción IVAD para poder acceder directamente a su rutina de servicio.



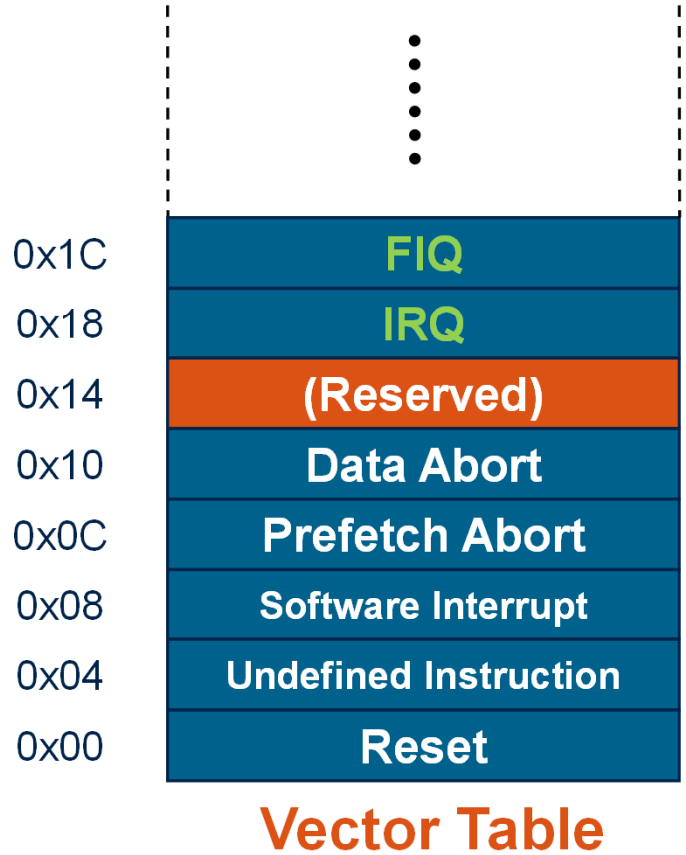
IVAD en ARM es una instrucción de salto que coloca al PC en la primera instrucción de la rutina de servicio

Exception Handling

- **When an exception occurs (/FIQ=0 o /IRQ=0), the ARM:**
  - Copies CPSR into SPSR_<mode>
  - Sets appropriate CPSR bits:
    - Changes to ARM state
    - Changes to related mode
    - Disables IRQ
    - Disables FIQ (only on fast interrupts)
  - Stores the return address in LR_<mode>
  - Sets PC to vector address
- **To return, exception handler needs to:**
  - Restore CPSR from SPSR_<mode>
  - Restore PC from LR_<mode>

    (more about this later…)

This can only be done in ARM state.

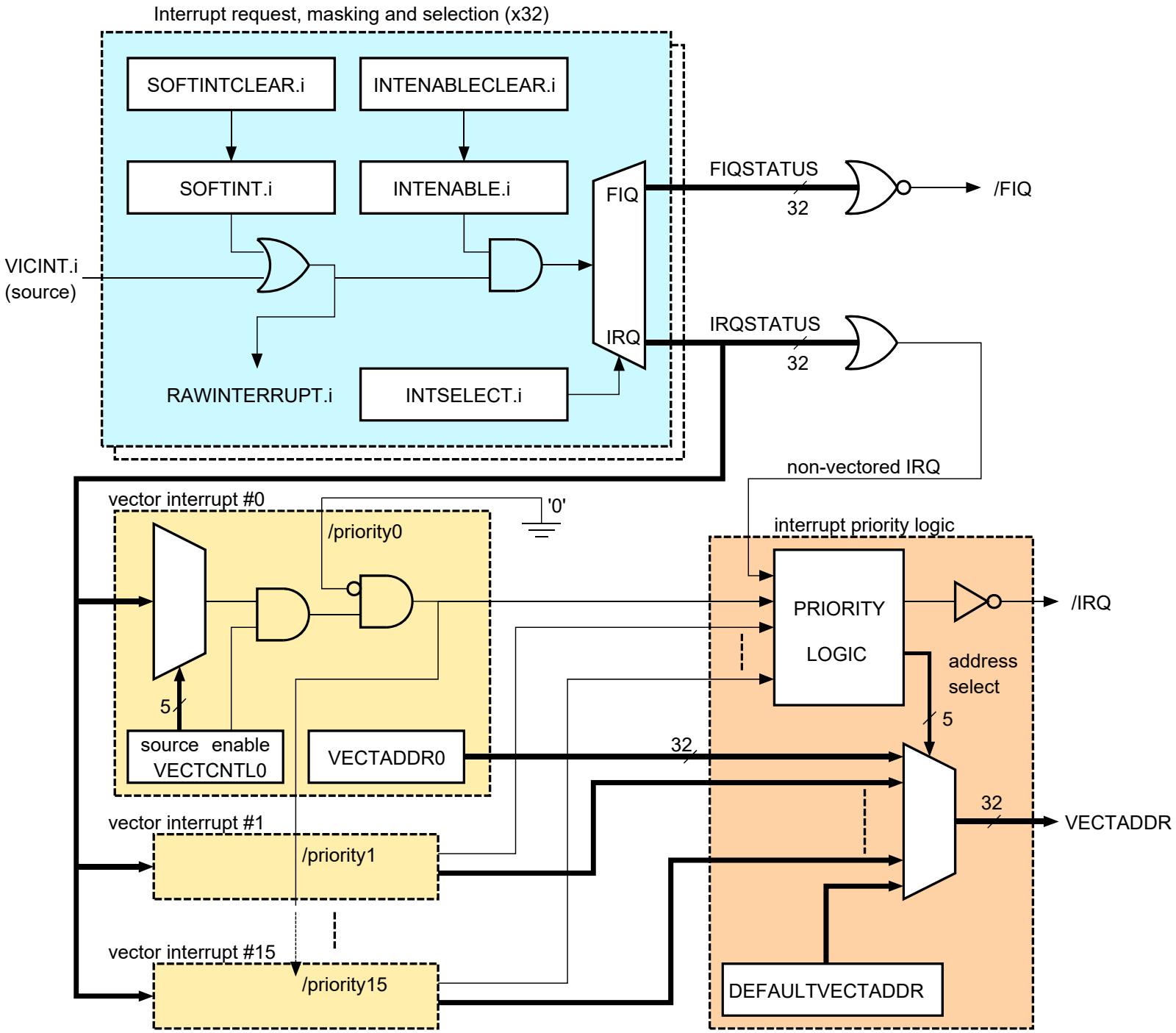


VIC is in charge of changing /IRQ or /FIQ bits and storing the IVAD of the current interrupt request into one of its registers VICVectAddr .

Vectored interrupt controller (VIC)

- Features
  - 32 interrupt inputs
  - 16 IRQ vectors
  - Default vector for non-vectored IRQs
  - Individual mask and software request
  - Interrupts can trigger IRQ of FIQ (selectable)
  - Priority chain
    - Vector #0: highest priority
    - …
    - Vector #15: lowest priority

VIC diagram


Interrupt request, masking and selection (x32)
SOFTINTCLEAR.i
INTENABLECLEAR.i
SOFTINT.i
INTENABLE.i
FIQ
IRQ
VICINT.i
(source)
RAWINTERRUPT.i
INTSELECT.i
FIQSTATUS
32
/FIQ
IRQSTATUS
32
non-vectored IRQ
vector interrupt #0
'0'
/priority0
5
source enable
VECTCNTL0
VECTADDR0
interrupt priority logic
PRIORITY
LOGIC
/IRQ
address
select
5
32
32
VECTADDR
vector interrupt #1
/priority1
vector interrupt #15
/priority15
DEFAULTVECTADDR

LPC2103. Interrupt mapping

Source	VICINT #
WDT	0
TIMER0	4
TIMER1	5
UART0	6
UART1	7
I2C0	9

Source	VICINT #
SPI0	10
SPI1/SSP	11
PLL	12
RTC	13
EINT0	14
EINT1	15

Source	VICINT #
EINT2	16
ADC	18
I2C1	19
TIMER2	26
TIMER3	27

VIC registers I

- Control (32-bits, one bit per VIC input, see interrupt mapping):
  - VICIntEnable: 1: Enable INT, 0: no action
  - VICIntEnClr: 1: Disable INT, 0: no action
  - VICIntSelect: 1: request as FIQ, 0: request as IRQ

  Interrupt request can be forced by setting bits in the SoftInt register:
  - VICSoftInt: 1: Set INT request, 0: no action
  - VICSoftIntClr: 1: Clear INT request, 0: no action

  (rarely used, may be useful for debugging purposes)

- Status (32-bits, one bit per VIC input , see interrupt mapping):
  - VICRawIntr: 1: Interrupt active, 0: no interrupt
  - VICIRQStatus: 1: Interrupt request made as IRQ, 0: no request
  - VICFIQStatus: 1: Interrupt request made as FIQ, 0: no request

VIC registers II

- Vectors:
  - VICDefVectAddr: Holds the ISR address for non-vectored and spurious IRQs
  - VICVectAddr0 to VICVectAddr15: Holds the ISR address of the corresponding vector
  - VICVectCntl0 to VICVectCntl15 (vector control):
    - Bits [4:0] Input channel selection (See interrupt mapping)
    - Bit 5 1: Vector enabled, 0: Vector disabled
- Vector reading:
  - VICVectAddr: Holds the address of the ISR for the current IRQ

  This register is read automatically by putting the following instruction in the ARM vector table:

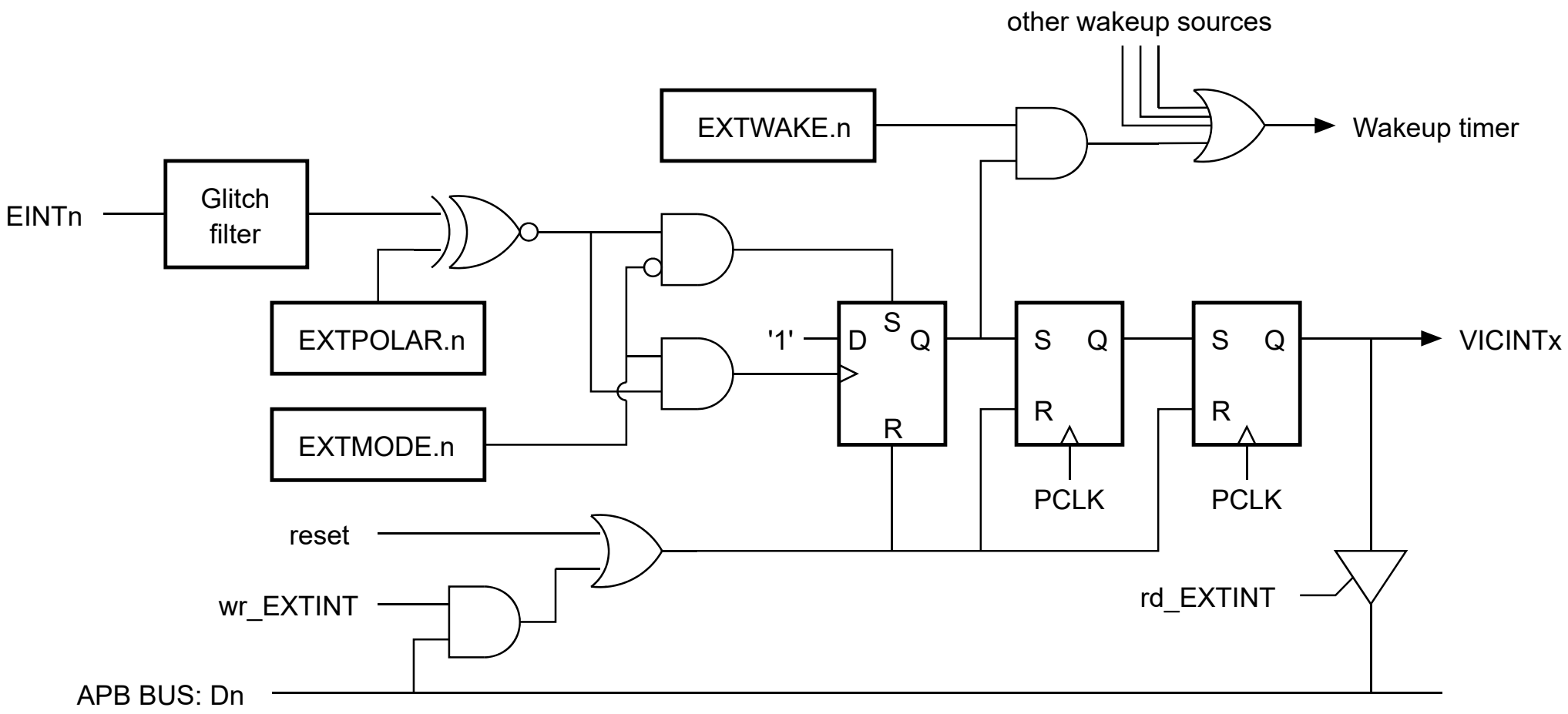
  0x18: 0xE51FFFF0 LDR pc, [pc, #-0xFF0] ; IRQ handler

  This instruction reads the PC from address 0xFFFFF030 (VICVectAddr register)
  - Writing any value to VICVectAddr updates the priority logic of the VIC. This has to be done at the end of ISRs (End Of Interrupt).

LPC2103. External interrupts

- Features
  - Part of the System Control Block peripheral
  - 3 interrupt inputs: EINT0, EINT1, EINT2
  - Level or edge sensitive
  - Any polarity
  - Can awake the microcontroller from power-down modes

LPC2103 External Interrupt Diagram



LPC2103. External Interrupt Registers

- EXTINT (8-bit): External Interrupt flags
  - Bits 0,1,2: EINT0, EINT1, EINT2

  Read: 1: active interrupt request, 0: no interrupt

  Write: 1: Clear interrupt flag 0: no action

  (These flags have to be cleared explicitly by the ISR for edge-mode ints.)
- EXTMODE (8-bit): External Interrupt mode
  - Bits 0,1,2: EXTMODE0, EXTMODE1, EXTMODE2

  0: Level sensitive 1: Edge sensitive
- EXTPOLAR (8-bit): External interrupt polarity
  - Bits 0,1,2: EXTPOLAR0, EXTPOLAR1, EXTPOLAR2

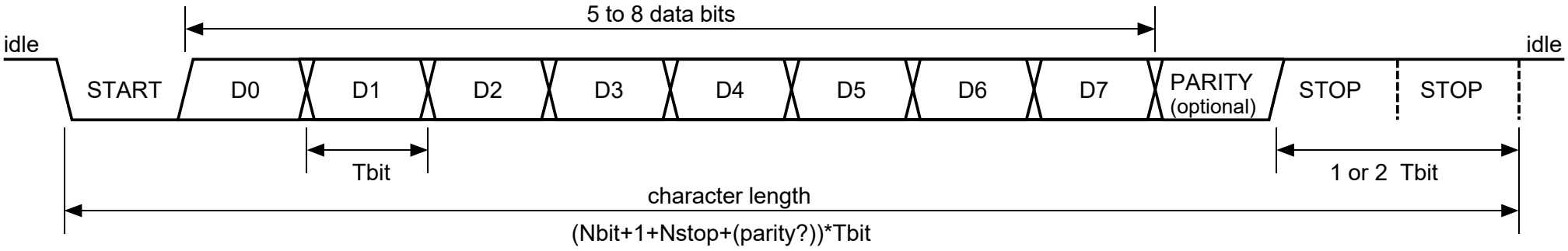
  0: Low level or falling edge 1: High level or rising edge
- INTWAKE (16-bit): Wakeup interrupts
  - Bits 0,1,2: EXTWAKE0, EXTWAKE1, EXTWAKE2
  - Bit 15: RTCWAKE

  1: The interrupt will awake the processor, 0: no wakeup

INDEX

- Introduction
- System Control Block
- GPIO
- Vectored Interrupt Controller (VIC)
- UART
- Timers

Asynchronous serial communications



- Asynchronous: no clock transmitted
  - Requires accurate clocks (relative error less than 3%) in both receiver and transmitter
- Character oriented
  - LSB first
  - Delimited by Start and Stop bits
  - Stop bits: minimum time with line high between characters
- Common data rates (data rate = 1/Tbit):

  300 bps, 1200 bps, 9600 bps, 19200 bps, 38400 bps, 115200 bps

Asynchronous serial communications

Data signals (DTE):

- TXD: Transmitted data
- RXD: Received data

Flow control (data block):

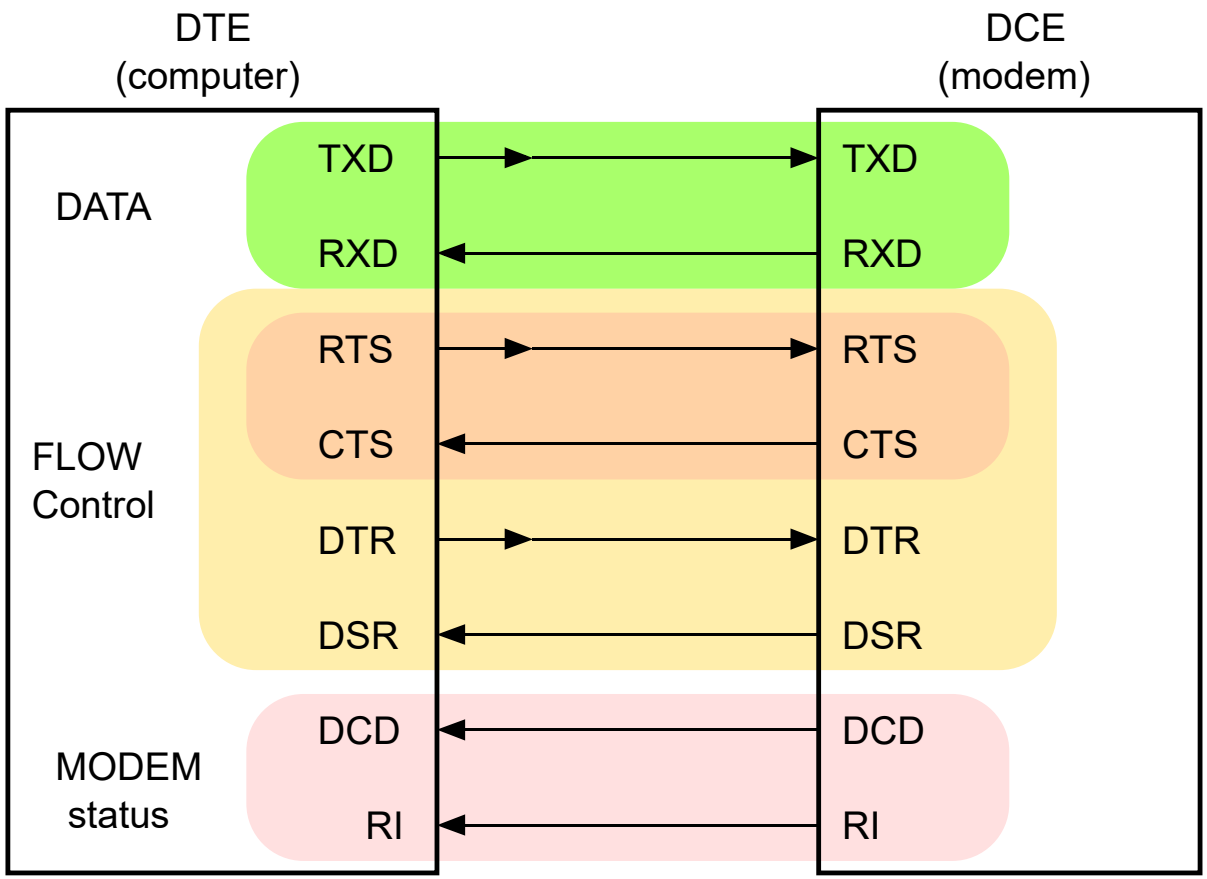
- RTS: Request to send
- CTS: Clear to send

Flow control (session):

- DTR: Data terminal ready
- DSR: Data set ready

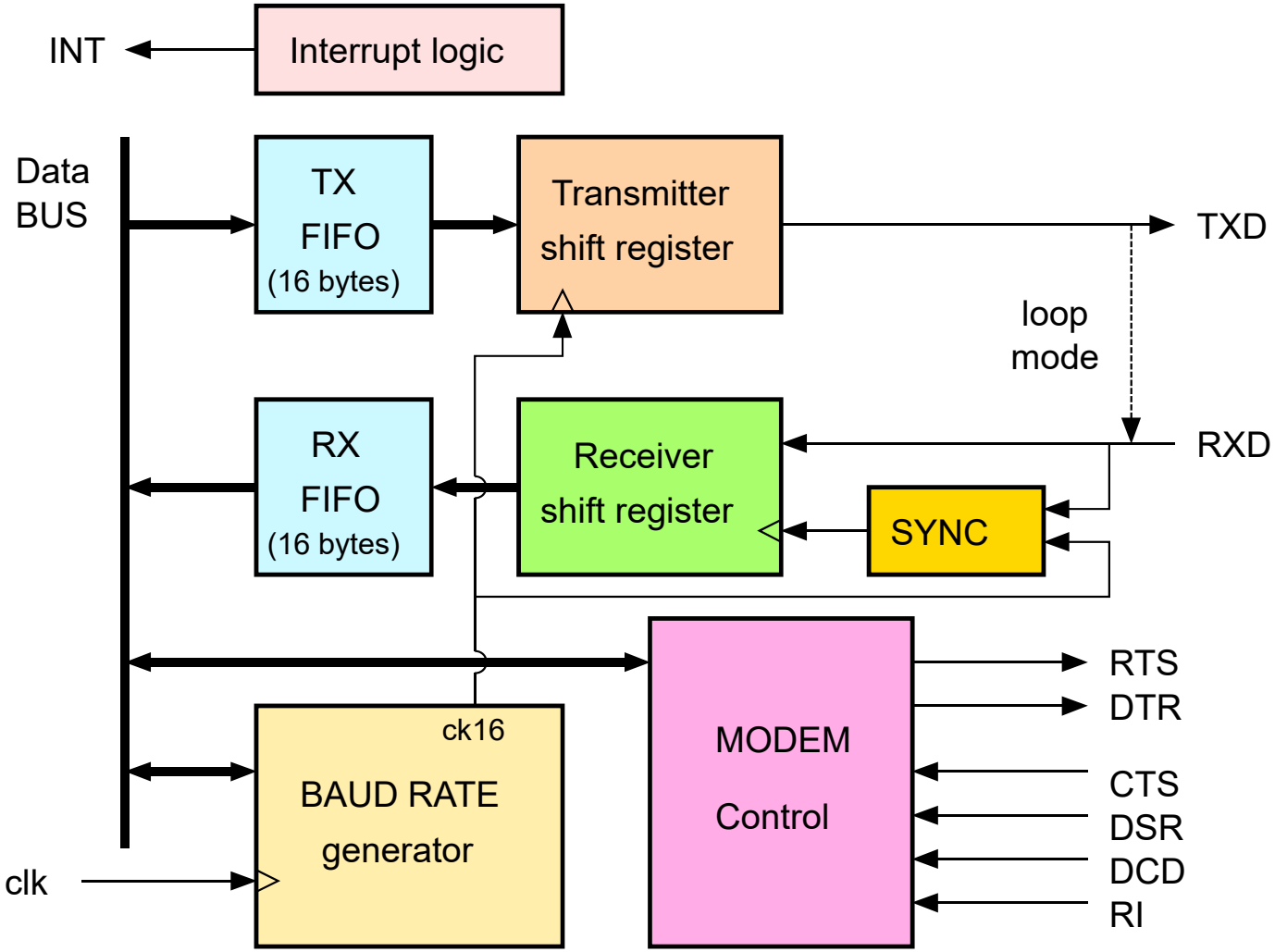
Modem status

- DCD: Data carrier detect
- RI: Ring indicator



Many serial ports only include the data lines and, maybe, a minimal flow-control (RTS,CTS)

UART 16550. Block diagram



- UART: Universal Asynchronous Receiver-Transmitter
- 16550: Industry standard. Used in PCs and many MCUs (LPC2xxx)
- 8-bit peripheral, banked register set

UART 16550 Register map

- RBR: Receiver Buffer Register
- THR: Transmitter Holding Reg.
- IER: Interrupt Enable Register
- IIR: Interrupt Identification Reg.
- FCR: FIFO Control Register
- LCR: Line Control Register
- MCR: Modem Control Register
- LSR: Line Status Register
- MSR: Modem Status Register

- DLM:DLL: Divisor Latch

	DLAB=0		DLAB=1	
addr.	RD	WR	RD	WR
0	RBR	THR	DLL	
1	IER		DLM	
2	IIR	FCR	IIR	FCR
3	LCR (DLAB is here)			
4	MCR			
5	LSR	-	LSR	-
6	MSR	-	MSR	-
7	Scratch			

- DLAB is only relevant for addresses 0 and 1
- In the LPC2xxx the register address are word aligned (multiply address by 4)

UART 16550 Registers

- RBR and THR. These registers hold the data received (RBR) and sent to transmitter (THR)
- Even with FIFOs disabled there are 1-byte buffers in front of shift registers
- With FIFOs enabled RBR and THR are used as FIFO_RX read and FIFO_TX write registers, respectively

UART 16550 Registers

- FCR: FIFO control register (write only)

Bit 0: FIFO enable: Enables both TX and RX FIFOs when 1

Bit 1: RX FIFO Reset: Flush RX FIFO when written as 1 (bit not stored)

Bit 2: TX FIFO Reset: Flush TX FIFO when written as 1 (bit not stored)

Bits 3 to 5: Reserved

Bits [7:6]: RX interrupt trigger level:

FCR[7:6]	Trigger level
00	1 byte
01	4 bytes
10	8 bytes
11	14 bytes

An RX data available interrupt is requested when the RX FIFO holds the same or more bytes than the programmed trigger level .

UART 16550 Registers

- LCR: Line Control Register

Bits [1:0]: Character Length

LCR[1:0]	Char. Len.
00	5 bits
01	6 bits
10	7 bits
11	8 bits

Bit 2: Number of Stop bits: 0: 1 stop bit, 1: 2 stop bits (1.5 bits for 5-bit characters)

Bit 3: Parity enable: 0: no parity, 1: parity added

Bit 4: Even parity: 0: Odd parity 1: Even parity

Bit 5: Stick parity: 0: normal parity 1: replace parity bit with LCR[4]

Bit 6: Break control: 0: no break 1: TXD is forced low (Break state)

Bit 7: DLAB: Select which register to access for address 0 and 1. DLAB=1 allows the access to the divisor latch of the Baud rate generator

UART 16550 Registers

- LSR: Line Status Register:

Bit 0: Data Ready: 1: there is data ready for read from RBR, 0: no data

Bit 1: Overrun Error: 1: The RX FIFO was full and data was lost, 0: no error

Bit 2: Parity Error: 1: Wrong parity in the received character, 0: no error

Bit 3: Framing Error: 1: The Stop bit(s) was 0 (wrong data rate?), 0: no error

Bit 4: Break interrupt: 1: The RXD line was low for more time than the duration of the character, 0: no break

Bit 5: THR Empty: 1: The TX FIFO or THR are not full (a character can still be present in the shift register),

0: The TX FIFO or THR are full

Bit 6: TX Empty: 1: Both the TX FIFO and the shift register are empty

0: At least there is a character in the shift register

Bit 7: Error in RX FIFO: 1: There are errors in the data stored in the RX FIFO

0: Data without errors

UART 16550 Registers

- IER: Interrupt Enable Register

  Bit 0: Rx Data Available interrupt enabled when 1

  Bit 1: Transmitter Holding Register Empty interrupt enabled when 1

  Bit 2: Receiver Line Status interrupt enabled when 1

  (When an OE, PE, FE error or a Break state are detected)

  Bit 3: Modem Status Change interrupt enabled when 1

  (When CTS, DRS, DCD or RI change value with respect to last MSR read)

  Bits 4-7: Reserved

UART 16550 Registers

- IIR: Interrupt Identification Register

Bit 0: Pending INT: 0: Pending interrupt, 1: no interrupt

Bits [2:1]: Interrupt cause

IIR[2:1]	Priority	Cause	Clearing Action
00	Lowest	Modem Status	MSR read
01	Second	THR Empty	THR write
10	Third	RX Data Available	RBR read
11	Highest	RX Line Status	LSR read

Bit 3: RX timeout:

Set along RX Data Available when some characters have been stored in the RX FIFO without reaching the programmed trigger level for more than 4 character times

Bits 4,5: Always 0

Bits 6,7: 00 if FIFOs are disabled, 11 if FIFOs are enabled

UART 16550 Registers

- MCR: Modem Control Register

Bit 0:	/DTR: Data Terminal Ready	0: DTR High,	1: DTR Low
Bit 1:	/RTS: Request to Send	0: RTS High,	1: RTS Low
Bit 2:	/OUT1 (GPIO)	0: OUT1 High,	1: OUT1 Low

(OUT1 is used in PCs to further enable the UART interrupt through a tristate gate outside the UART)

Bit 3:	/OUT2 (GPIO, not used)	0: OUT2 High,	1: OUT2 Low
Bit 4:	Loop mode:	0: Normal mode	1: Loop mode

In loop mode the TXD and RXD signals are connected together inside the UART. Any character transmitted is sent to the receiver. Also, DTR,RTS,OUT1 and OUT2 are tied to DSR,CTS,RI and DCD, respectively.

Bits 5 to 7: Reserved

UART 16550 Registers

- MSR: Modem Status Register

Bit 0:	Delta CTS	1: CTS changed	0: no change
Bit 1:	Delta DSR	1: DSR changed	0: no change
Bit 2:	Delta RI	1: RI changed	0: no change
Bit 3:	Delta DCD	1: DCD changed	0: no change
Bit 4:	/CTS, Clear to Send,	1: CTS Low	0: CTS High
Bit 5:	/DSR, Clear to Send,	1: DSR Low	0: DSR High
Bit 6:	/RI, Clear to Send,	1: RI Low	0: RI High
Bit 7:	/DCD, Clear to Send,	1: DCD Low	0: DCD High

- Delta bits are set when the corresponding input pin changes its level with respect to that of the last MSR read.
- Any delta bit set can trigger a Modem Status interrupt

UART 16550 Registers

- Baud Rate Divider (accessed when DLAB=1)

  DLM: Most significant byte of the 16-bit divider

  DLL: Least significant byte of the 16-bit divider

- Baud Rate Calculation:

$$f_baud = f_clk/(16*divider)$$

- f_clk is PCLK for LPC2xxx MCUs (and 1.8432 MHz for PCs)
- A divider value of zero has the same effect as divider=1

INDEX

- Introduction
- System Control Block
- GPIO
- Vectored Interrupt Controller (VIC)
- UART
- Timers

LPC2103 Timers/Counters

- 4 Timers:
  - Timer0, Timer1: 32-bit
  - Timer2, Timer3: 16-bit
- Each Timer can include up to:
  - 4 capture inputs, sensitive to signal edges. These signals can also be used as an external clock (counter mode)
  - 4 matching outputs, that can set, clear, or toggle a pin on matching events. They can also generate PWM waveforms

  Some lines are unavailable:

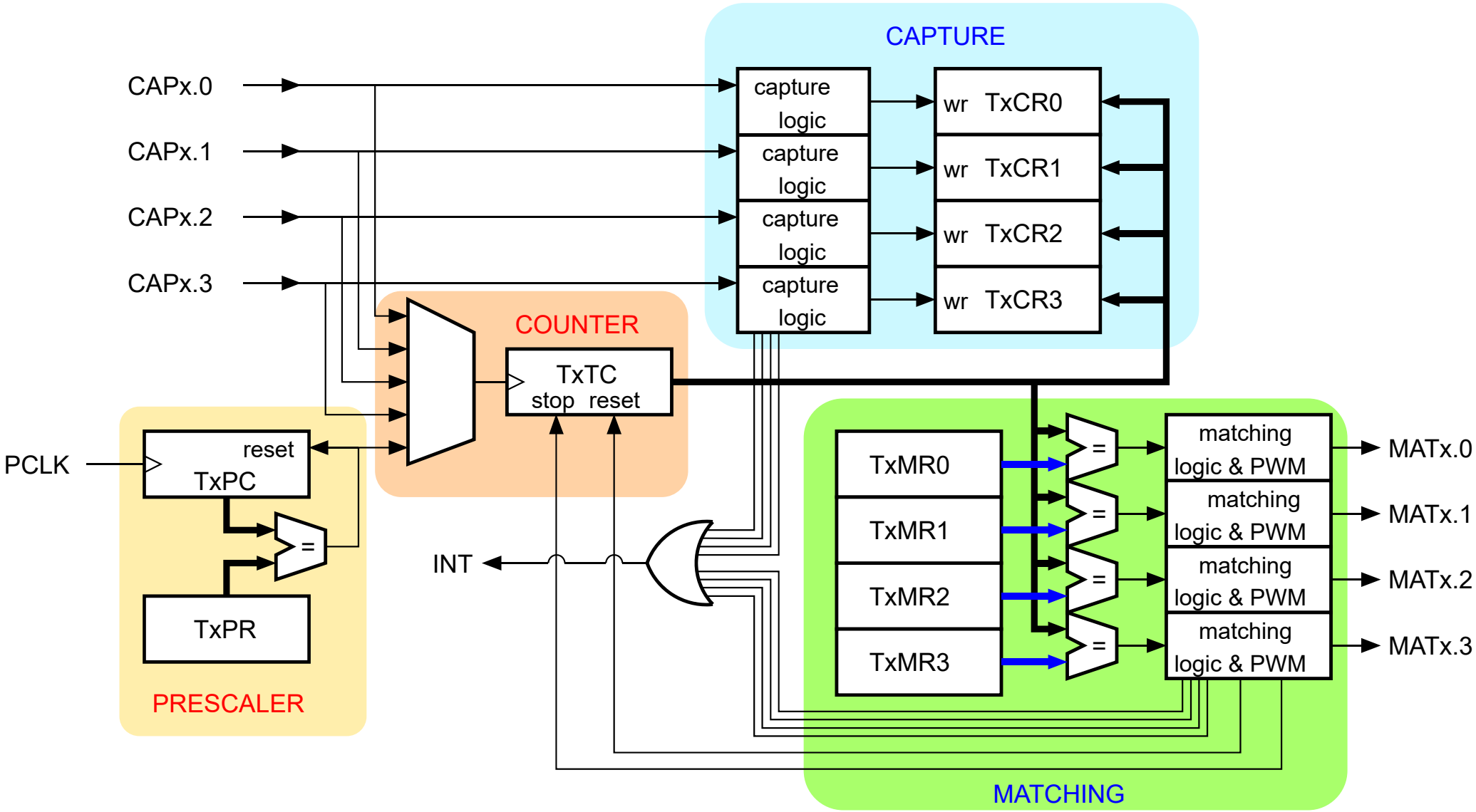
  Timer 0: no CAP0.3, no MAT0.3

  Timer 2: no CAP2.3

  Timer 3: no CAP3.0, no CAP3.1, no CAP3.2, no CAP3.3
- All timers have the same architecture and register set (but many registers have different widths: 32 or 16 bits)

(In the following slides replace ‘x’ with the timer number. e.g. TxTC -> T1TC)

LPC2xxx Timers. Block Diagram



LPC2xxx Timers. Registers. Control

- TxTC: Timer counter
- TxPC: Prescaler counter
- TxPR: Prescaler register:

  TxPC increments at PCLK rate until the value stored in TxPR is reached. Next cycle, TxPC resets and TxTC increments (if timer mode). The clock frequency of TxTC is then: PCLK/(TxPR+1)

- TxTCR: Timer Control Register:

Bit 0: Enable	1: Timer enabled,	0: Timer disabled
Bit 1: Reset	When 1 both TxTC and TxPC are reset. The reset state will last until this bit is written with 0	
Bits 2-7:	Reserved	

LPC2xxx Timers. Registers. Control

- TxCTCR: Counter Control Register

Bits [1:0]: Mode select (00 after reset)

TxCTCR[1:0]	Mode
00	Timer. Increment TxTC on PCLK divided by (TxPR+1)
01	Counter. Increment TxTC on rising edges
10	Counter. Increment TxTC on falling edges
11	Counter. Increment TxTC on both edges

Bits [3:2]: Clock source in counter mode

TxCTCR[3:2]	Clock source
00	CAPx.0
01	CAPx.1
10	CAPx.2
11	CAPx.3

Bits 4-7: Reserved

LPC2xxx Timers. Registers. Capture

- TxCR0, TxCR1, TxCR2, TxCR3 : Capture registers. When a capture event happens the value of TxTC is copied into one of these registers
- TxCCR: Capture Control register (3 bits per channel)

  Bits [2-0]: CAPx.0 control:

  - Bit 0: Capture on rising edges if 1
  - Bit 1: Capture on falling edges if 1
  - Bit 2: Generate Interrupt on capture if 1

  Bits [5-3]: CAPx.1 control (same coding)

  Bits [8-6]: CAPx.2 control (same coding)

  Bits [11-9]: CAPx.3 control (same coding)

  Bits [15-12]: Reserved

LPC2xxx Timers. Registers. Matching

- TxMR0, TxMR1, TxMR2, TxMR3 : Matching registers. When TxTC matches the value of one of these registers several possible actions can take place
- TxMCR: Matching Control register (3 bits per match register)

Bits [2-0]: MR0 control:

Bit 0: Generate interrupt on matching if 1

Bit 1: Reset TxTC on matching if 1

Bit 2: Stop TxTC on matching if 1

Bits [5-3]: MR1 control (same coding)

Bits [8-6]: MR2 control (same coding)

Bits [11-9]: MR3 control (same coding)

Bits [15-12]: Reserved

LPC2xxx Timers. Registers. Matching

- TxEMR: External Match register (matching actions on pins)

Bit 0: Logic value of MATx.0

Bit 1: Logic value of MATx.1

Bit 2: Logic value of MATx.2

Bit 3: Logic value of MATx.3

Bits [5:4]: MATx.0 Control

00	Do nothing
01	Clear MATx.0
10	Set MATx.0
11	Toggle MATx.0

Bits [7:6]: MATx1 Control (same coding)

Bits [9:8]: MATx2 Control (same coding)

Bits [11:10]: MATx3 Control (same coding)

Bits [15-12]: Reserved

LPC2xxx Timers. Registers. PWM

- PWMxCON: PWM Control

  Bit 0: MATx.0 PWM mode enabled if 1

  Bit 1: MATx.1 PWM mode enabled if 1

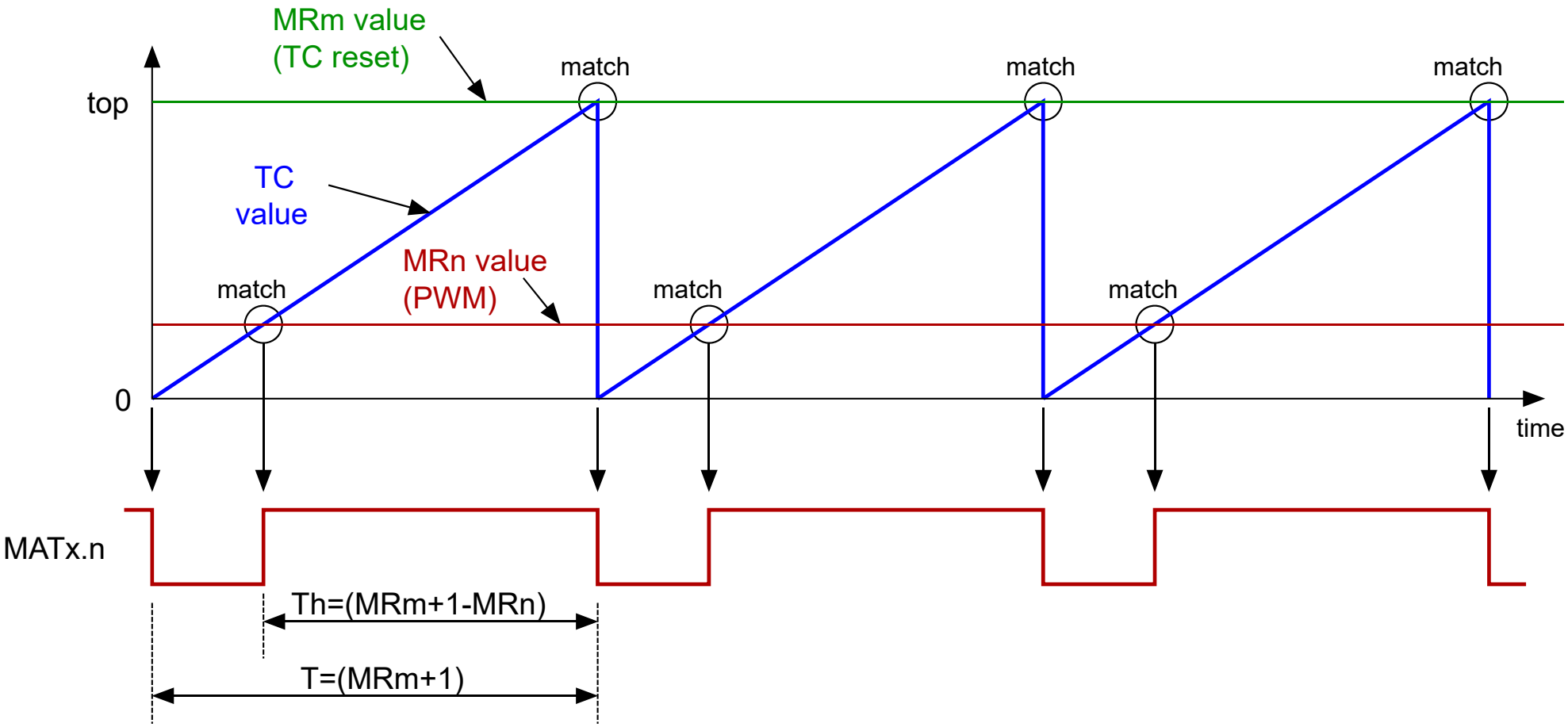
  Bit 2: MATx.2 PWM mode enabled if 1

  Bit 3: MATx.3 PWM mode enabled if 1

  Bits [31-4]: Reserved

- PWM modulation is achieved by turning the MATx.i pin LOW when TxTC is 0 and HIGH when a match happens
- When PWM mode is selected for a pin its TxEMR bits are ignored
- Another TxMR register has to be used to define the cycle period (by resetting TxTC)

LPC2xxx Timers. Registers. PWM timing



- If TxMRn=0 => MATx.n always High
- If TxMRn=TxMRm => Single-cycle High pulse on MATx.n
- If TxMRn>TxMRm => MATx.n always Low

LPC2xxx Timers. Registers. Interrupts

- TxIR: Interrupt Register

  Bit 0: TxMR0 match interrupt

  Bit 1: TxMR1 match interrupt

  Bit 2: TxMR2 match interrupt

  Bit 3: TxMR3 match interrupt

  Bit 4: Capture to TxCR0 interrupt

  Bit 5: Capture to TxCR1 interrupt

  Bit 6: Capture to TxCR1 interrupt

  Bit 7: Capture to TxCR1 interrupt

These bits have to be cleared by program by writing a mask to TxIR: (1: Clear interrupt, 0: no effect)

- No interrupt on timer overflow is available. Use a match interrupt with TxMRn=0 if needed